Zend

System Documentation

Architecture, trust boundaries, clients, relay flow, and operational notes

Strydr Silverberg

April 19, 2026

A concise technical guide to the overall Zend system: what components exist, how they fit together, what security properties they aim to provide, and where the current limits are.

Contents

1 Overview **3**
- 1.1 What Zend Is . . . 3
- 1.2 Current Repository Shape . . . 3
- 1.3 Primary Design Idea . . . 3

2 System Architecture **4**
- 2.1 Relay-Backed Share Flow . . . 4
  - 2.1.1 Flow Summary . . . 4
- 2.2 Direct CLI Flow . . . 4

3 Trust Boundaries and Security Model **6**
- 3.1 What the Relay Should Not Learn . . . 6
- 3.2 What the Relay Still Learns . . . 6
- 3.3 Blob Integrity . . . 6
- 3.4 Operational Safety Properties in the Current Relay . . . 7

4 Protocol Notes **8**
- 4.1 Shared Blob Format . . . 8
- 4.2 Why the Shared Format Matters . . . 8

5 Current Limits **9**
- 5.1 What the System Already Does Well . . . 9
- 5.2 What Is Intentionally Not Claimed . . . 9
- 5.3 Known Technical Tradeoffs . . . 9

6 Verification and Maintenance **10**
- 6.1 Current Verification Story . . . 10
- 6.2 Practical Verification Commands . . . 10

Chapter 1

Overview

1.1 What Zend Is

Zend is an encrypted file transfer project built mostly in Zig. In its current form, the repository contains two related transfer paths:

1. a relay-backed share flow used by the web app and relay-aware CLI
2. a direct peer-to-peer CLI flow over TCP

The relay-backed flow is the center of the current project. Files are encrypted on the client, stored on the relay as ciphertext, and later downloaded and decrypted client-side. The decryption key is carried in the URL fragment rather than sent to the relay as part of the HTTP request.

1.2 Current Repository Shape

Path	Role
`apps/relay`	Zig HTTP relay for encrypted blob storage and download
`apps/web`	Next.js frontend for upload and download flows
`apps/cli`	Zig CLI for relay flows and direct peer-to-peer transfer
`packages/protocol`	Shared blob framing, metadata, integrity, and parsing
`packages/crypto`	ChaCha20, Poly1305, and AEAD implementation
`packages/compress`	Huffman encoder / decoder used opportunistically
`packages/wasm`	Zig-to-WASM build used by the browser client

Table 1.1: Top-level system components

1.3 Primary Design Idea

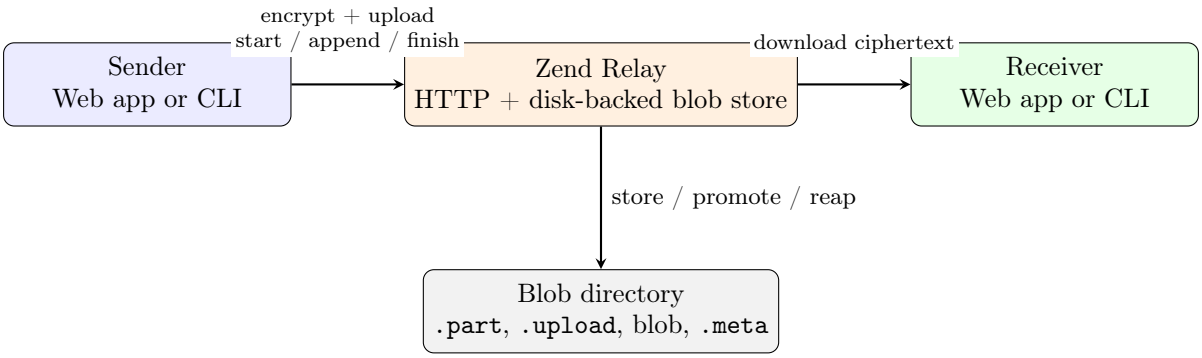
The relay is intended to sit outside the trust boundary for file contents. It stores and serves encrypted bytes, but it should not possess the key required to decrypt them.

> **Note:** This is not a metadata-hiding design. The relay can still observe timing, file existence, ciphertext size, request paths, and client IP information.

Chapter 2

System Architecture

2.1 Relay-Backed Share Flow



2.1.1 Flow Summary

1. The sender chooses a file.
2. The client generates a random 32-byte key locally.
3. The file is packetized, optionally compressed per chunk, encrypted, and tagged with a whole-file integrity record.
4. Ciphertext is uploaded to the relay in ordered append requests.
5. The sender shares a link of the form `/d/{id}#{key}`.
6. The recipient downloads the ciphertext blob.
7. The client decrypts the file and verifies final size and SHA-256 integrity.
8. The relay deletes the stored blob after a successful download.

2.2 Direct CLI Flow

The CLI also supports a separate direct-transfer mode where one machine connects directly to another over TCP.

- The receiver listens on a TCP port.
- The sender connects directly.
- The peers perform an X25519-based handshake.
- File chunks are encrypted and transferred over the established channel.

Note: The direct CLI flow is conceptually separate from the relay-backed flow. The relay-backed path uses a locally generated shared-file key embedded in the URL fragment, while the direct path uses an interactive key exchange.

Chapter 3

Trust Boundaries and Security Model

3.1 What the Relay Should Not Learn

In the relay-backed design, the relay should not learn:

- the plaintext file contents
- the file encryption key
- the decrypted filename, unless a client leaks it elsewhere

This is achieved by encrypting the file before upload and placing the decryption key in the URL fragment, which browsers do not send as part of HTTP requests.

3.2 What the Relay Still Learns

The relay can still observe:

- upload and download timing
- blob size
- client IP information
- request paths and route usage
- whether a blob exists, has expired, or has already been consumed

3.3 Blob Integrity

The shared protocol code includes an integrity packet containing:

- the final plaintext size
- the SHA-256 digest of the final plaintext

The decrypt path only treats a file as complete after:

1. all frames have been processed in order
2. authenticated decryption has succeeded for each packet
3. the final plaintext size matches
4. the final plaintext hash matches

3.4 Operational Safety Properties in the Current Relay

The current relay implementation enforces several narrow but important safety rules:

- upload appends must arrive strictly in sequence
- uploads are capped by total size and per-append size
- completion requires the correct upload token
- deletion requires the correct blob token
- rate limits are applied per IP and per route class
- completed blobs and abandoned uploads are reaped on TTL

Warning: These properties improve safety and abuse resistance, but they do not amount to a formal security audit. Anyone deploying the project for serious use should still review the code and threat model directly.

Chapter 4

Protocol Notes

4.1 Shared Blob Format

The relay-backed path uses a shared encrypted blob format implemented in Zig and reused by the CLI and WASM targets.

The blob format currently includes:

- encrypted metadata packet with filename and total plaintext size
- encrypted chunk packets
- optional Huffman compression applied on a chunk-by-chunk basis when it reduces size
- encrypted integrity packet with plaintext size and SHA-256 digest
- encrypted DONE packet

4.2 Why the Shared Format Matters

This shared implementation is one of the strongest consistency points in the current codebase:

- the browser client and relay-aware CLI speak the same format
- integrity behavior is not duplicated in separate language implementations
- protocol-level tests can cover behavior that matters to multiple clients at once

Chapter 5

Current Limits

5.1 What the System Already Does Well

- client-side encryption for relay-backed sharing
- relay-side storage of ciphertext rather than plaintext
- single-use relay download semantics
- cleanup of expired blobs and incomplete uploads
- shared protocol logic across CLI and browser paths
- CI-backed test and build checks

5.2 What Is Intentionally Not Claimed

The current system is not presented as:

- an anonymity system
- a synchronized cloud drive
- a multi-user storage platform
- a formally audited cryptographic product
- a system with hidden access patterns or hidden metadata

5.3 Known Technical Tradeoffs

- The relay download route currently reads the full blob into memory before responding.
- The web app currently has its own UI-level file size cap.
- Configuration style still differs somewhat between the relay, the web app, and the CLI.
- The direct peer-to-peer CLI mode does not attempt NAT traversal.

Chapter 6

Verification and Maintenance

6.1 Current Verification Story

The repository now includes:

- RFC-vector tests for low-level crypto primitives
- protocol tests for round-trip behavior, tamper detection, and truncation handling
- relay utility tests for ID validation, cleanup helpers, and rate limiting
- end-to-end relay tests covering upload, finish, download, consume-on-read, and token-protected deletion
- CI jobs for Zig tests/checks and web lint/build

6.2 Practical Verification Commands

```
zig build test
zig build check
cd apps/web && pnpm lint && pnpm build
```

Tip: The project is at its best when the README, CI workflow, and these commands all tell the same story. If one of them changes, the others should be updated in the same pull request.